AWS S3 and IAM: Detailed Access-Control Lab Report

Younes Chemmaoui
Prepared: 7 November 2026
Source captures: 29 October - 7 November 2026

1. Objective and lab scope

I used this lab to understand how the active AWS principal, IAM identity policies and S3 bucket policies affect the same storage operations. The central question was why an IAM user could create a bucket but could not upload an object, while an assumed role could access objects in buckets governed by different policies.

The training environment supplied devuser, DeveloperGroup, the role configurations, and three lab buckets. I investigated those resources through the AWS Management Console. The report describes access attempts, policy inspection and explanations from my notes; it does not claim that I built the supplied IAM infrastructure.

The work has three stages: examine devuser permissions, switch to BucketsAccessRole and compare results, then investigate the bucket3 challenge and use the role named by its resource policy. The screenshots and notes show both unsuccessful operations and later successful operations.

2. Source material and presentation

This expanded report uses both the written lab observations and the longer working note. It includes all 38 images from the lab folder, including original screenshots, diagrams, annotations and repeated views. The files in assets/ are byte-for-byte copies with their original filenames; no image has been edited, redacted or cropped for this revision.

The captures span multiple training sessions and account identifiers. Their sequence explains the workflow, but they are not represented as one continuous Console session. The late screenshots add a successful bucket3 upload that the older written report had left unverified.

I distinguish observed results from the training instructions and explanatory figures. An instruction saying to download or upload a file is not itself proof of success. Where the screenshot supplies direct evidence, the discussion states what is visible. Illustrative CLI output and conceptual diagrams are identified separately.

3. Method

I attempted operations as devuser, inspected DeveloperGroupPolicy, switched roles, repeated object access, and compared the outcomes with the policies. The Console was the main test interface. No new AWS experiments were performed while preparing this publication, and no CloudTrail export or reproducible CLI transcript is claimed.

4. Results overview

The following matrix brings together the observations in the written report and the original captures. Allowed and Denied refer to the recorded Console behavior, not a newly executed automated test.

Active principal and operation	Recorded outcome	Evidence and interpretation
devuser: EC2 instance listing	Denied	EC2 dashboard and Instances captures show authorization errors.
devuser: IAM account summary	Denied	The IAM dashboard identifies iam:GetAccountSummary as unauthorized.
devuser: list IAM groups	Allowed	DeveloperGroup is visible after switching back to devuser.
devuser: create a bucket	Allowed	Creation-success capture; the group policy includes s3:CreateBucket.
devuser: upload a file	Denied	Upload summary reports one failed file and Access denied.
devuser: download bucket1 object	Denied	XML error names s3:GetObject and no identity-based allow.
devuser: assume BucketsAccessRole	Allowed	Switch-role form and later active-role header are captured.
BucketsAccessRole: download from bucket1	Allowed	Image2.jpg browser download is marked Done.
BucketsAccessRole: list IAM groups	Denied	IAM reports missing iam:ListGroups for the assumed-role request.
BucketsAccessRole: upload to bucket2	Allowed	Object listing and later Upload succeeded capture.
devuser: initial bucket3 upload	Denied in notes	The associated challenge picture is a policy-editor view, not an upload-result screen.
OtherBucketAccessRole: bucket3 upload	One file succeeded	Later summary shows Image2.jpg and Succeeded with the role active.

The key lesson is to review the identity, action, resource and all relevant policy paths together. The role identity policy explains bucket1 read access. The bucket2 resource policy explains an additional object-access path. The bucket3 policy names a different role, which is visible in the later successful upload capture.

A successful upload proves the captured operation. It does not establish that every S3 action is allowed, that every policy restriction was audited, or that an unseen grading workflow passed. Those limits remain explicit throughout the report.

5. Initial environment and intended access paths

The environment began with an IAM user in DeveloperGroup, a policy attached to that group, and three S3 buckets. The diagrams below are the original scenario figures retained with my notes. They describe the intended training topology, not resources that I independently provisioned.

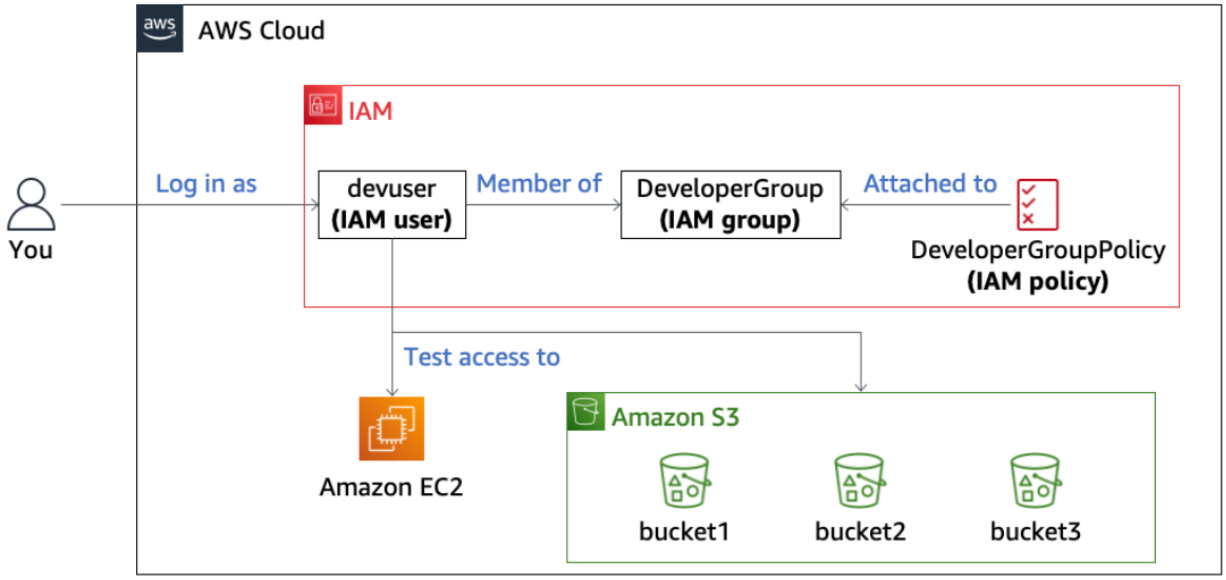


Figure 1. Original starting scenario: devuser, DeveloperGroup, its identity policy and the three training buckets.

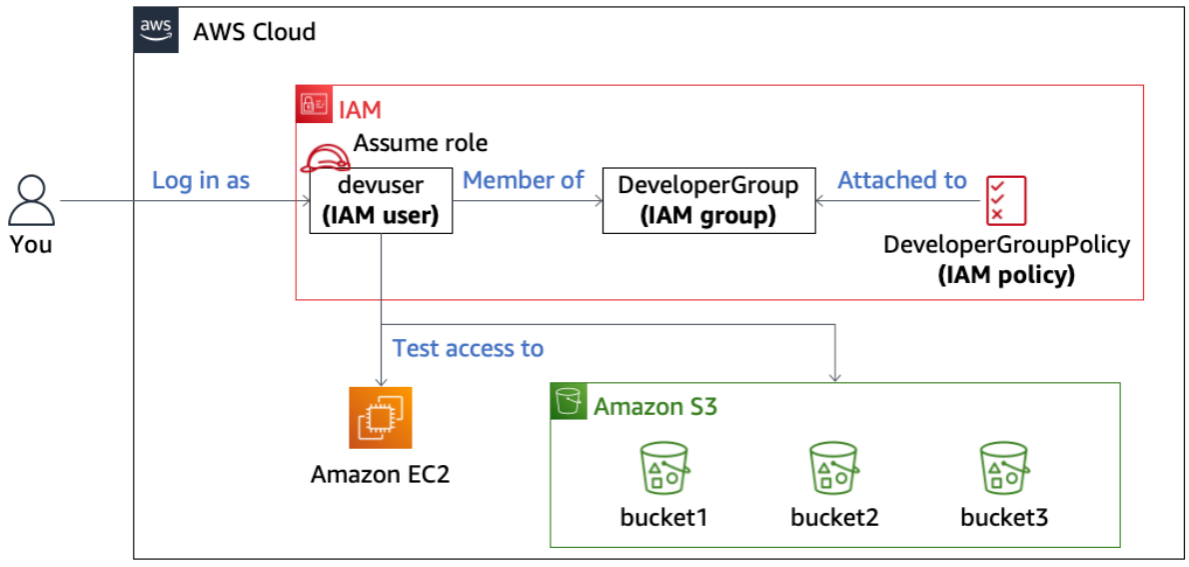


Figure 2. Original end-state scenario showing role assumption as an additional access path.

I treated the diagrams as a map of the exercise, then checked the actual outcomes through the Console. The important boundary is between the user identity and the assumed-role session. The bucket names and roles were supplied by the lab, while the access tests and policy analysis were the practical work.

6. Read-level access as devuser: EC2

After signing in as devuser, I explored EC2. The dashboard displayed repeated Access denied messages. The Instances view supplied a more specific authorization error. This is stronger evidence than simply seeing an empty list: the request was rejected, so the screen did not establish whether instances existed.

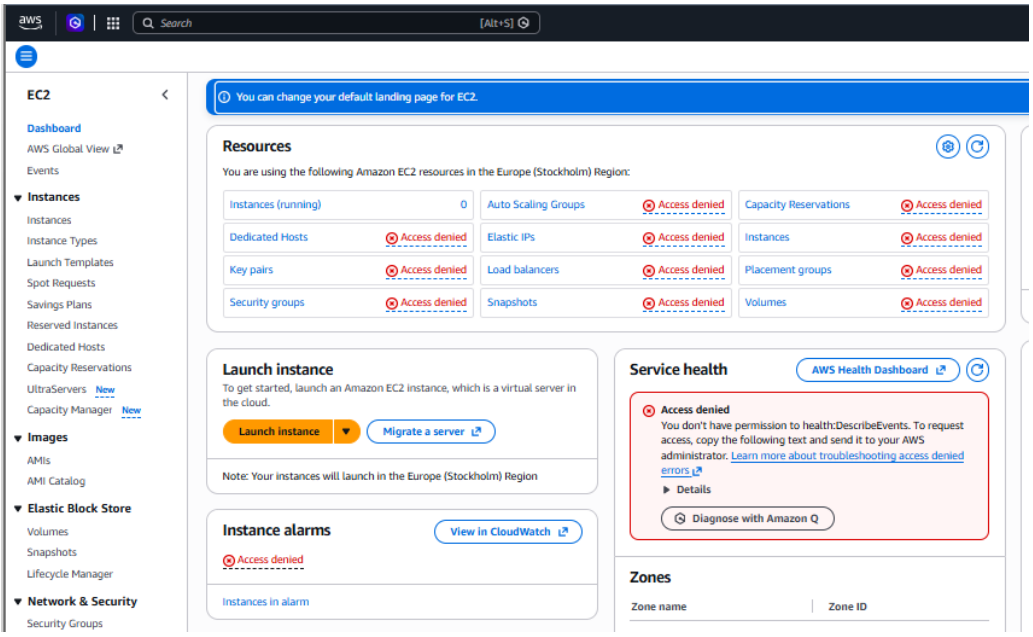


Figure 3. EC2 dashboard with multiple authorization errors in the original capture.

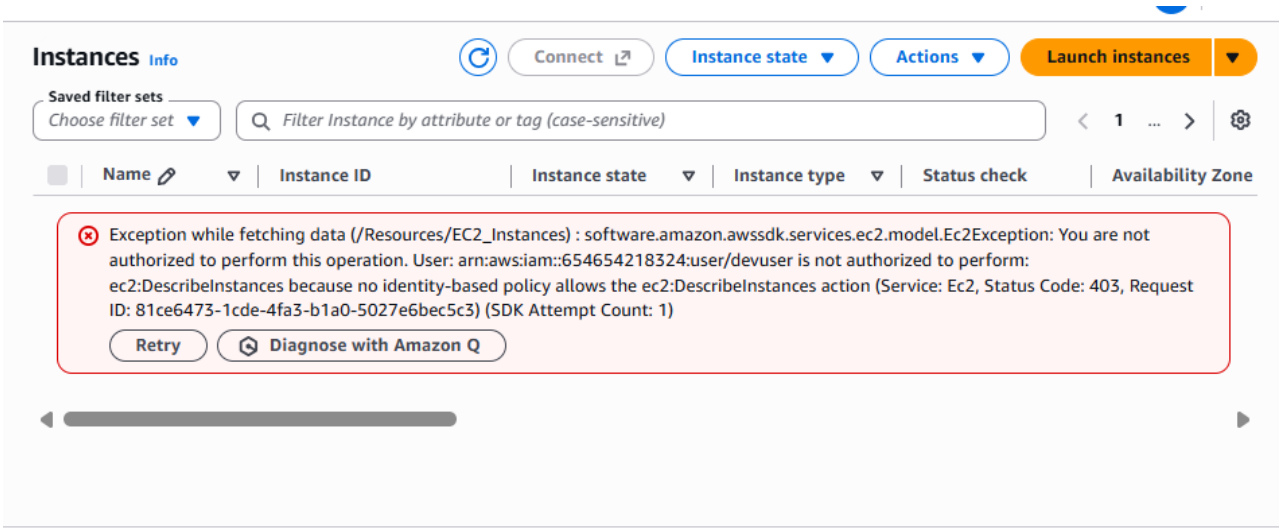


Figure 4. EC2 Instances request denied; the error identifies the requested action and active IAM user.

The working note also describes checking the key-pair selection required for an instance launch and cancelling the launch workflow. I do not claim to have launched an EC2 instance. These observations concern the captured requests and permissions, rather than a complete test of EC2 in every Region.

7. Read-level access as devuser: S3 and IAM

S3 displayed the three lab buckets, but listing bucket names did not mean that object contents were readable. In IAM, the account-summary view produced a separate missing-permission error. These screens helped separate service discovery, bucket listing and individual API permissions.

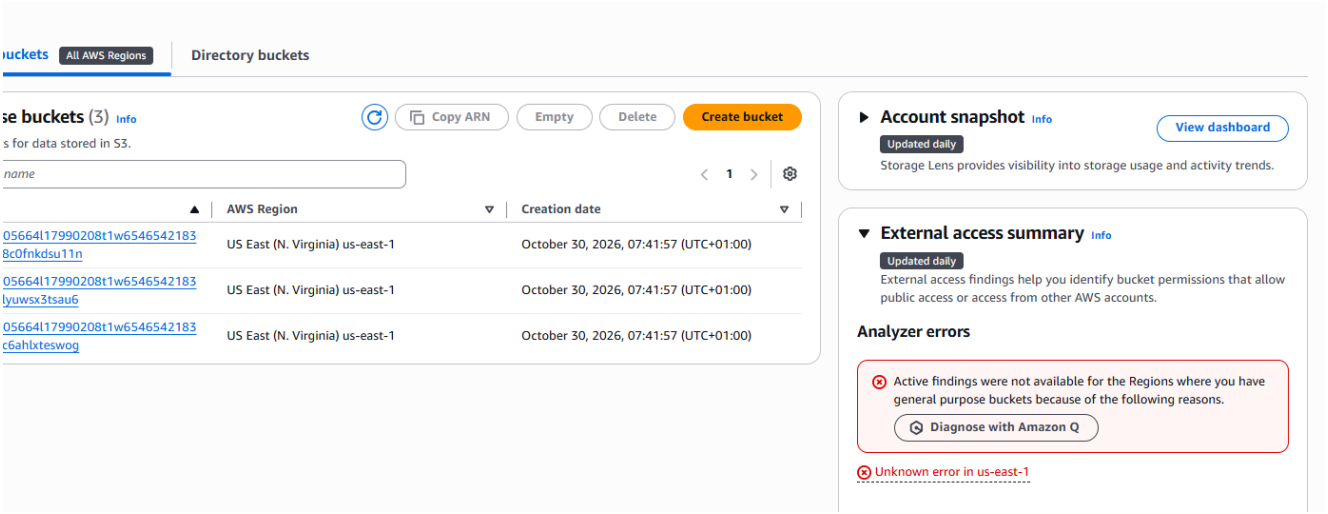


Figure 5. S3 bucket list and associated permission warnings.

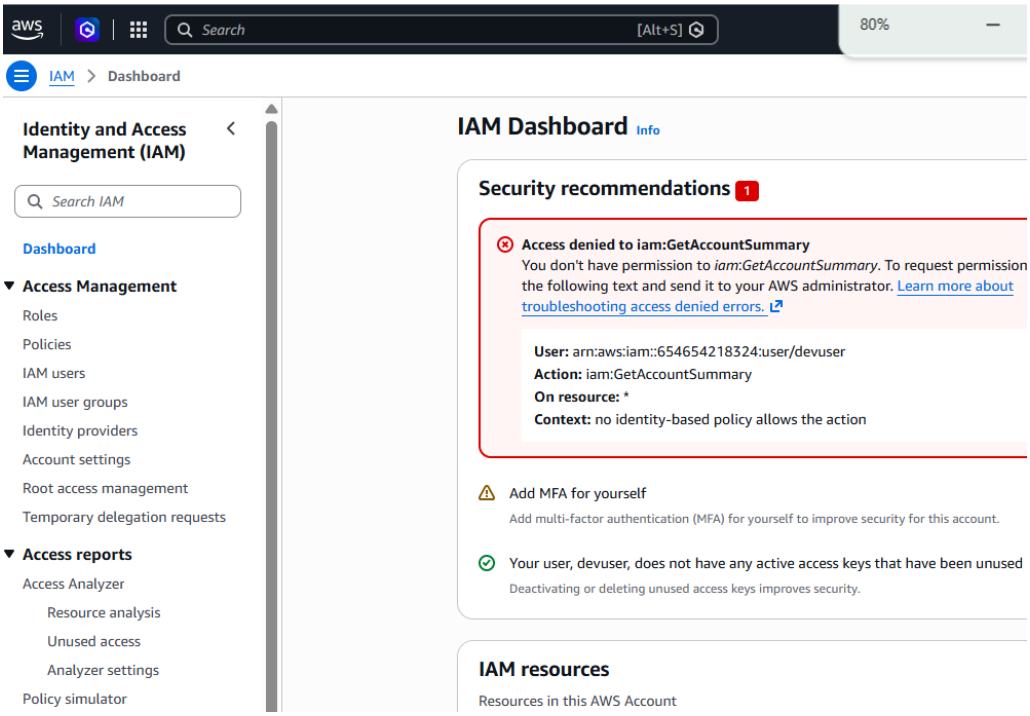


Figure 6. IAM dashboard identifies iam:GetAccountSummary as unauthorized for devuser.

The IAM summary error did not mean that every IAM function was unavailable. The same user could inspect the group and its policies. The distinction matters because IAM actions such as GetUser, GetGroupPolicy, ListGroups and GetAccountSummary are separately authorized.

8. DeveloperGroupPolicy and bucket creation

I inspected DeveloperGroup and the policy inherited by devuser. The captured action list contains selected CloudFormation, IAM and CloudWatch Logs reads, S3 bucket creation/listing/configuration actions, and sts:AssumeRole. Resource is broad in this supplied training policy. It is not presented as a production least-privilege recommendation.



Figure 7. Original group-policy capture, including s3:CreateBucket, s3:ListBucket and sts:AssumeRole, but no s3:PutObject.

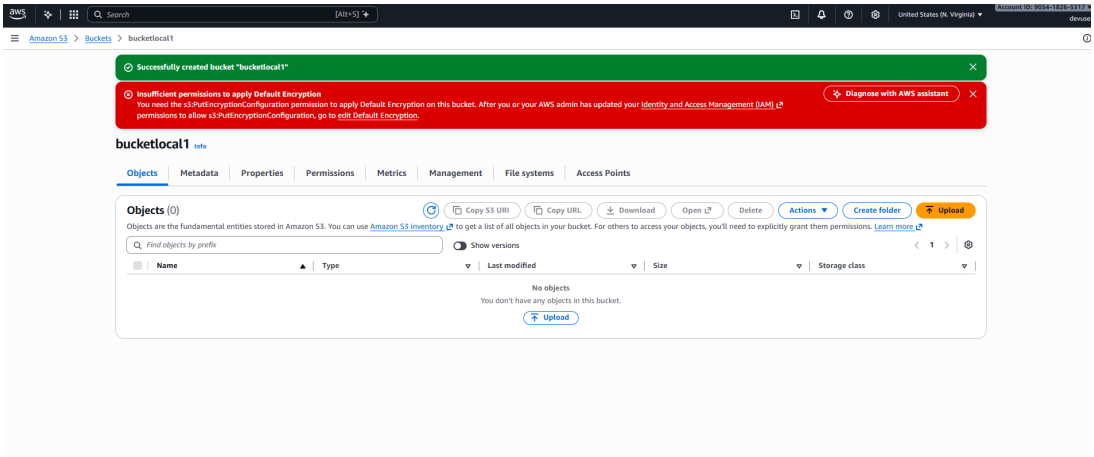


Figure 8. Later successful bucket-creation capture, together with a separate default-encryption permission warning.

The green creation banner and the encryption warning describe different operations. Successful creation does not establish permission to configure every bucket feature. The important observation for the next test was that s3:PutObject was absent from the displayed action list.

9. Object operations denied as devuser

The next tests showed why creating or listing a bucket is different from reading or writing an object. The upload result reports one failed image file and Access denied. Although the supplied procedure mentioned a policy JSON file, the captured failed upload is a PNG; I preserve that difference rather than claim that the requested file was used.

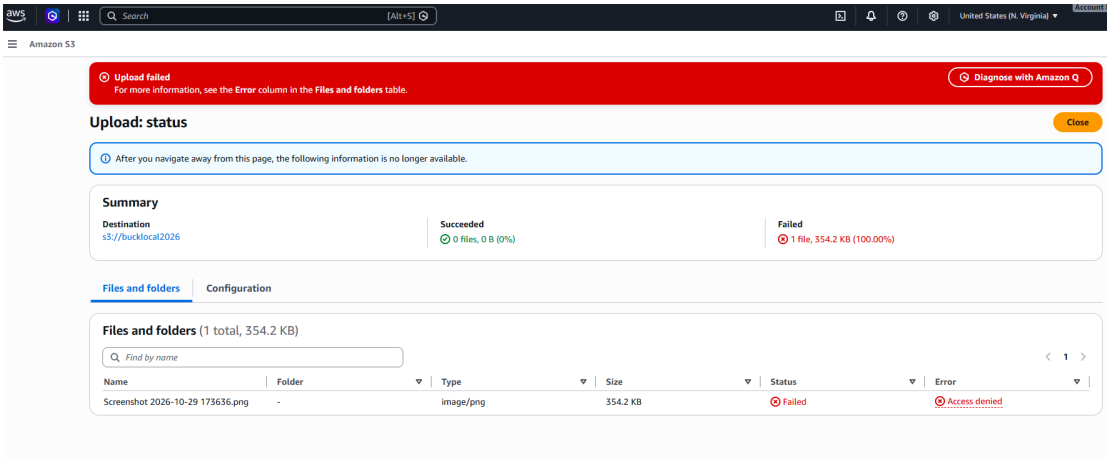


Figure 9. Original failed upload summary, including the file row and Access denied result.

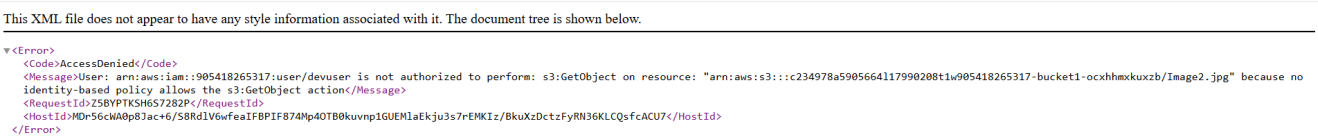


Figure 10. Original XML authorization-error capture associated with an attempted object read.

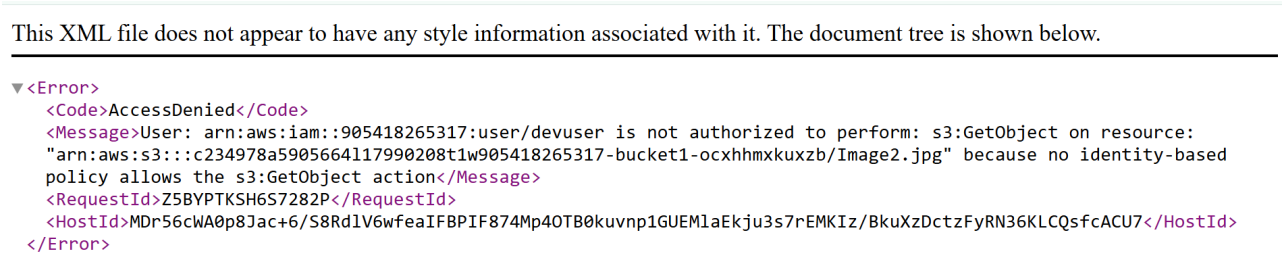


Figure 11. A second original XML capture explicitly reports denial of s3:GetObject for devuser.

The GetObject message states that no identity-based policy allows that action for the request. This supports the recorded read denial. Request and host identifiers are diagnostic context, not evidence of an successful download.

10. Selecting the role in the Console

I opened the identity menu and examined the role history. The two captures show the same menu at different stages, with annotations pointing to the previously used BucketsAccessRole entry and the Switch role control. Both original images are retained because they record the actual navigation used in the note.

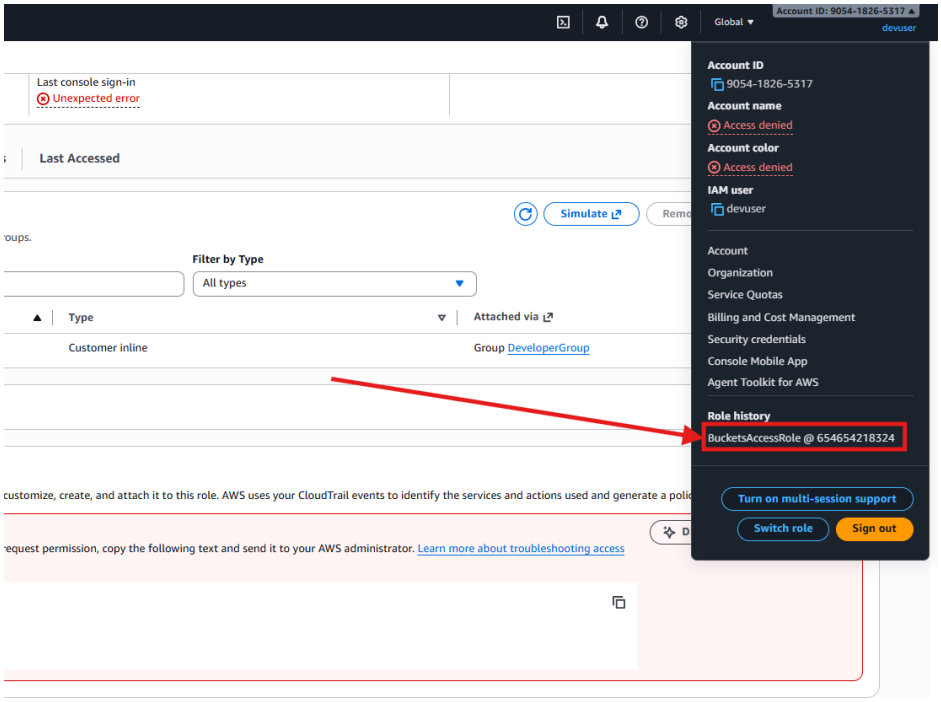


Figure 12. Role-history menu with the BucketsAccessRole entry highlighted.

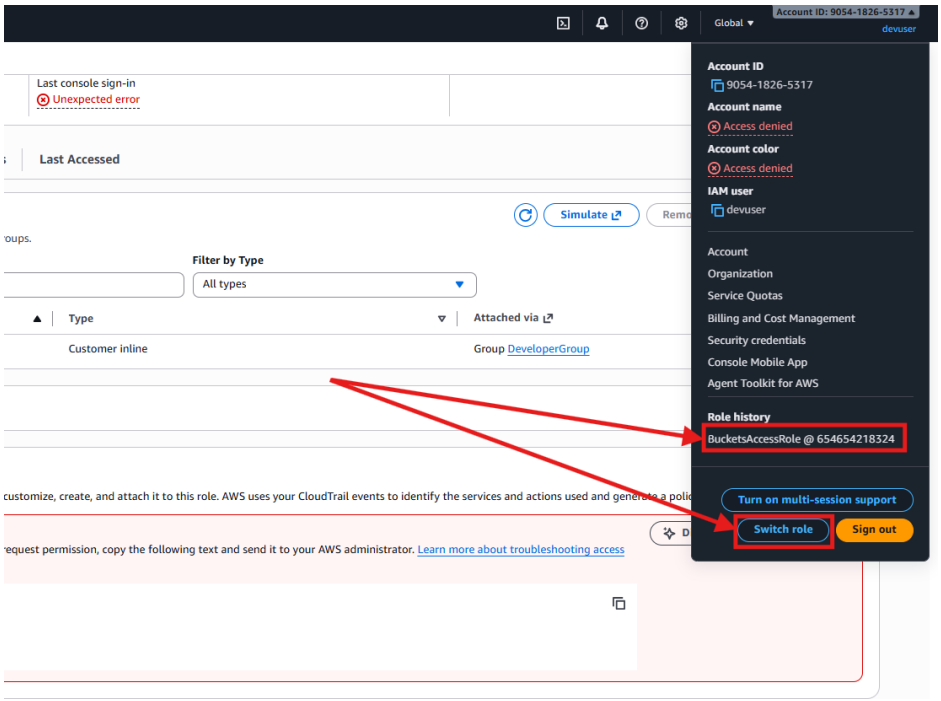


Figure 13. Annotated role-history menu also identifying the Switch role control.

Selecting a role is not the same as editing the user policy. It starts a role-session workflow. The resulting principal must be checked after switching, rather than assuming that the original user is still making the requests.

11. Switching into BucketsAccessRole

The switch-role form asked for the training account and role name. I used BucketsAccessRole as supplied by the lab. The later Console header shows that role as the active identity; the second original image adds an arrow to the header so the principal change is easy to locate.

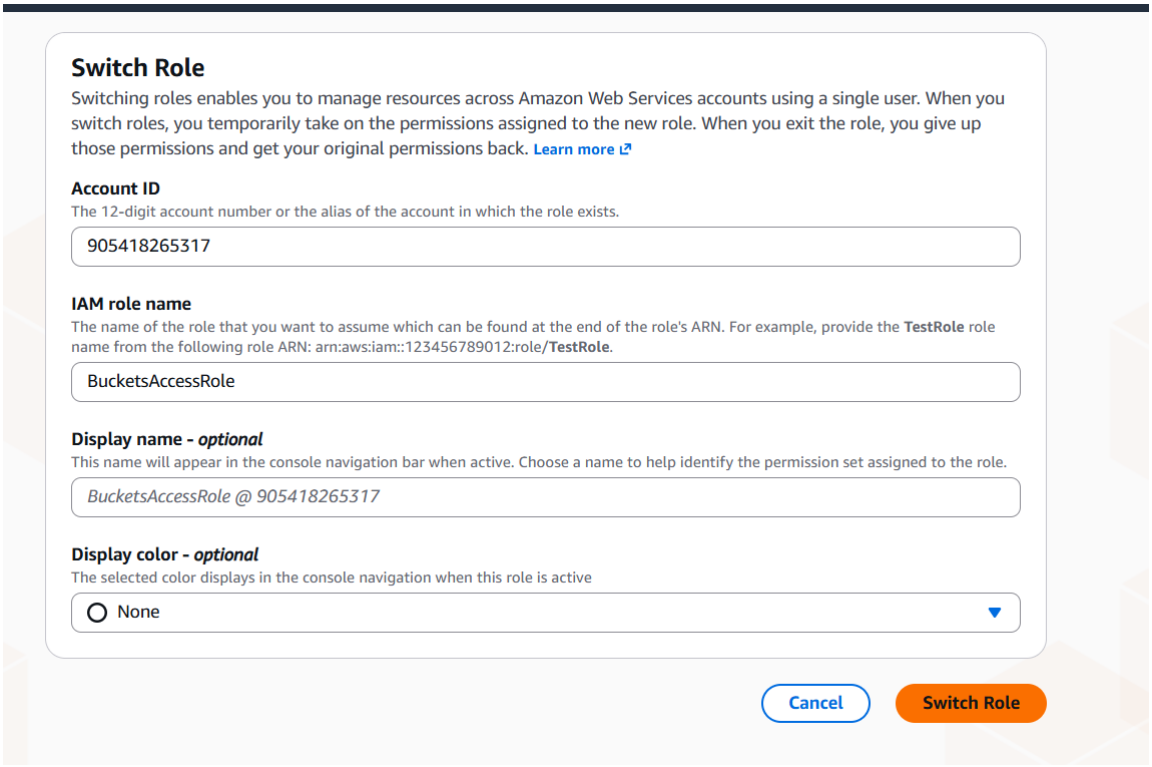


Figure 14. Original Switch Role form for BucketsAccessRole.

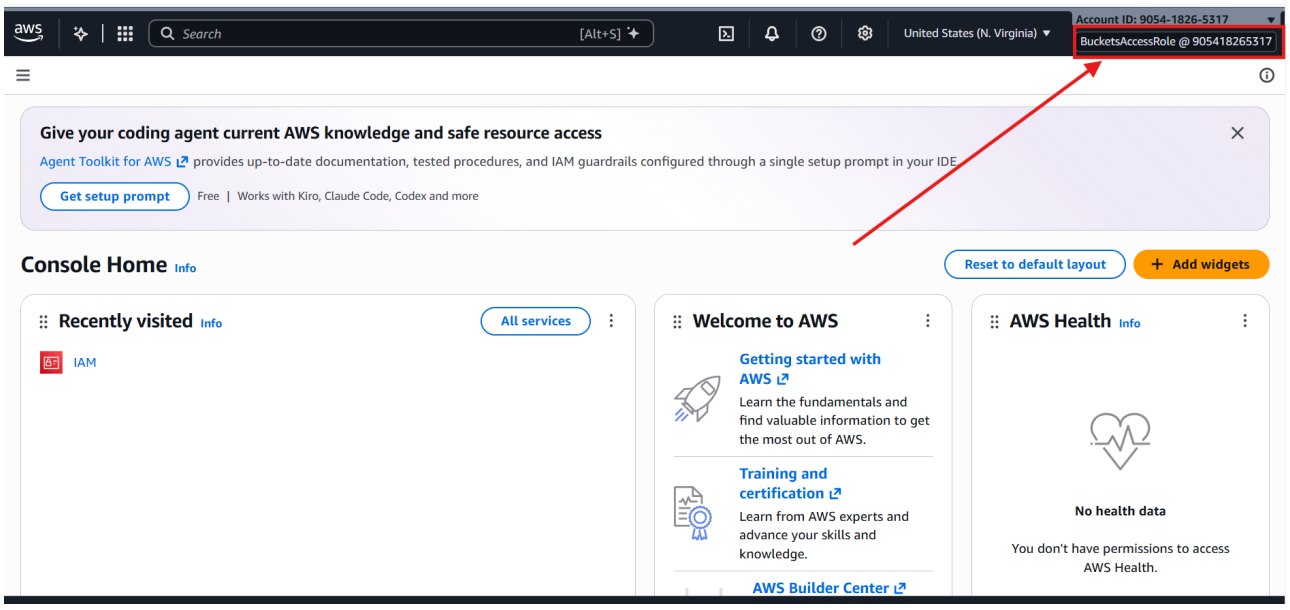


Figure 15. Annotated Console capture showing BucketsAccessRole as the active session.

This principal check is essential when comparing two authorization results. Without it, a successful or failed operation could be attributed to the wrong set of permissions. An additional unannotated role-session view is retained later in the original-image appendix.

12. Trust policy: who can assume the role

I reviewed the role trust relationship after inspecting its permissions. The captured policy names devuser as the trusted principal and sts:AssumeRole as the action. The two original versions show the same relationship. This document answers who can assume the role; it is not the policy that grants S3 object access after assumption.

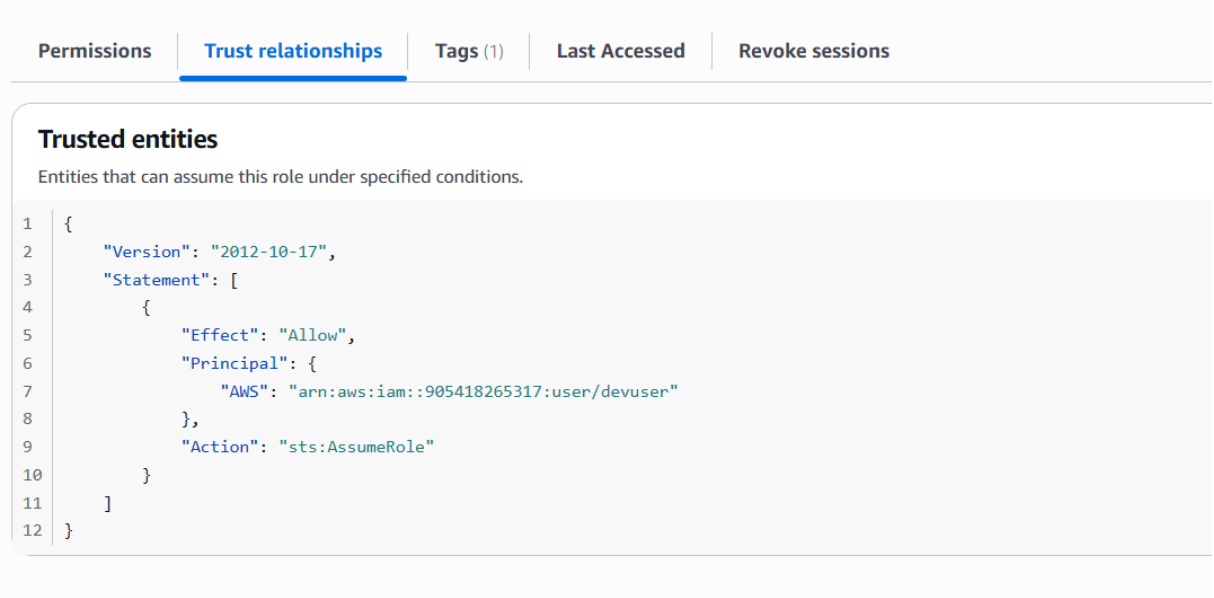


Figure 16. Original BucketsAccessRole trust-policy capture.

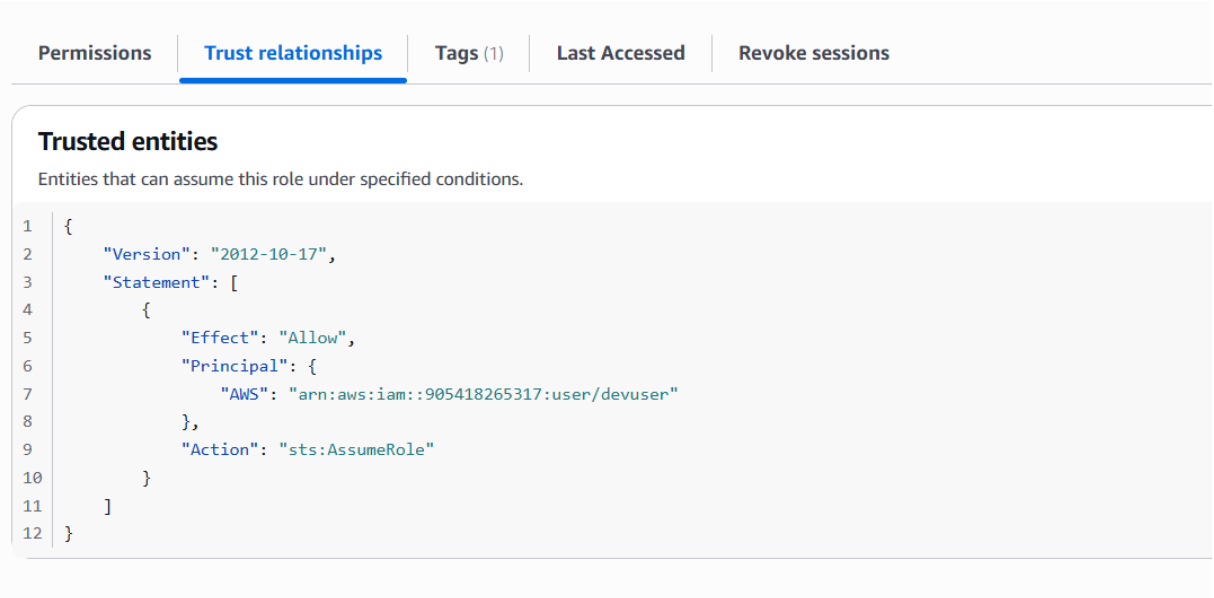


Figure 17. Second original trust-policy view retained in the working note.

The user policy also included sts:AssumeRole. In this lab, the visible trust and user permissions were consistent with the successful role switch. The screenshots do not constitute a full audit of every control that could affect role assumption in another account or environment.

13. Downloading from bucket1 as the role

After switching to BucketsAccessRole, I returned to S3 and retried the object-read workflow. The note identifies bucket1 and Image2.jpg. The download capture shows a browser notification for Image2.jpg with a completed status, while the Console header identifies the role session.

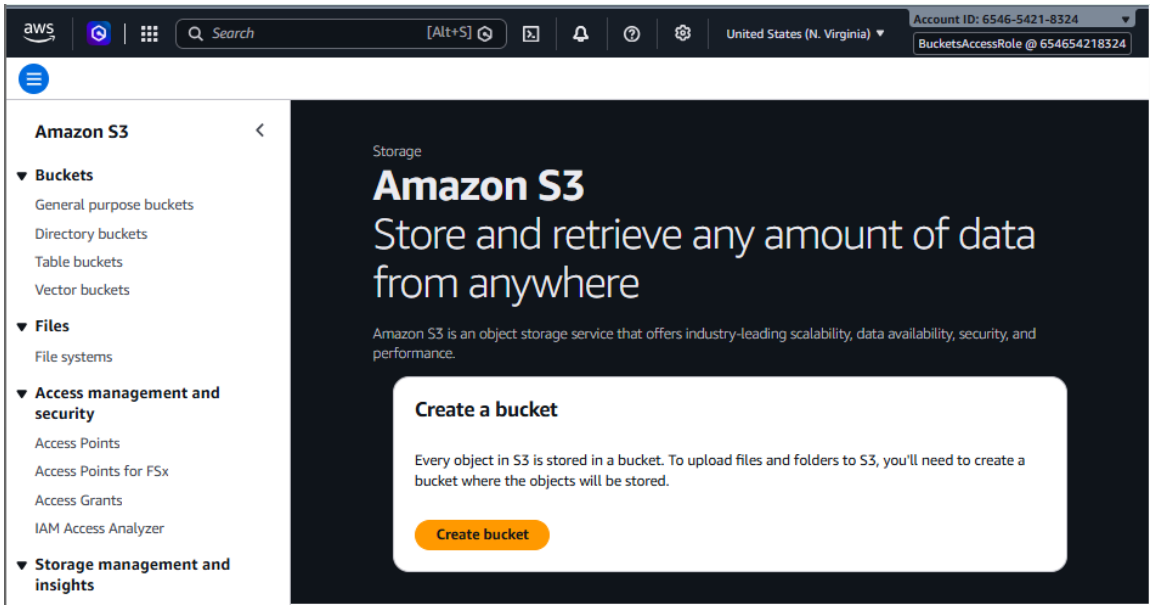


Figure 18. Original S3 Console landing view with BucketsAccessRole active.

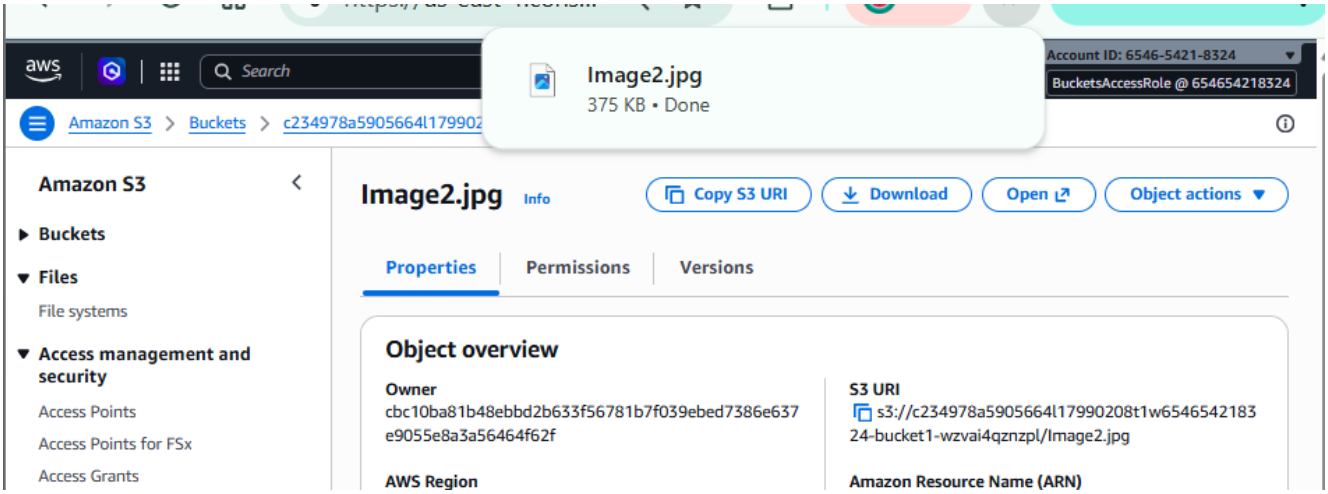


Figure 19. Original Image2.jpg capture with the completed browser download notification.

This differs from the earlier devuser XML denial. The result is consistent with the role policy that grants GetObject access to bucket1 objects. The observation establishes the captured download, not write permission on bucket1 or access to all other buckets.

14. IAM group visibility changed with the role

I also tested IAM after assuming the role. The User groups page reported an Access denied error for iam:ListGroups. I then switched back to devuser and could see DeveloperGroup again. The comparison shows that changing roles can remove an operation that the original user could perform.

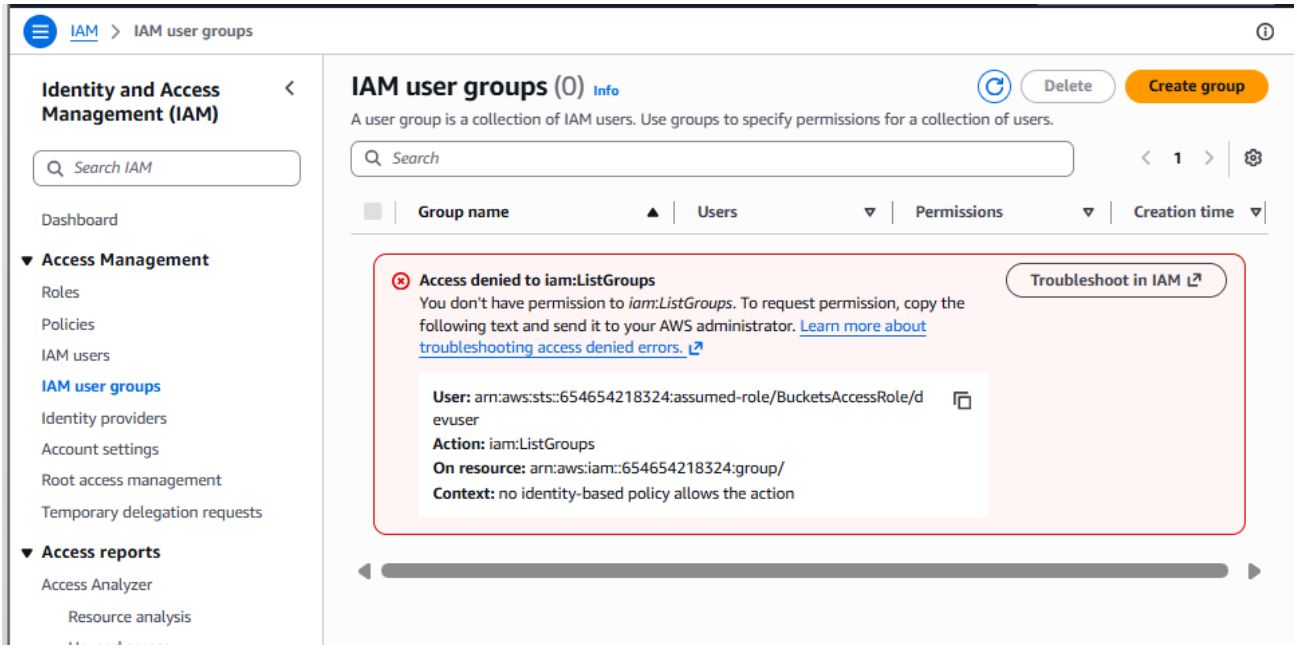


Figure 20. IAM group-list request denied for the assumed-role session.

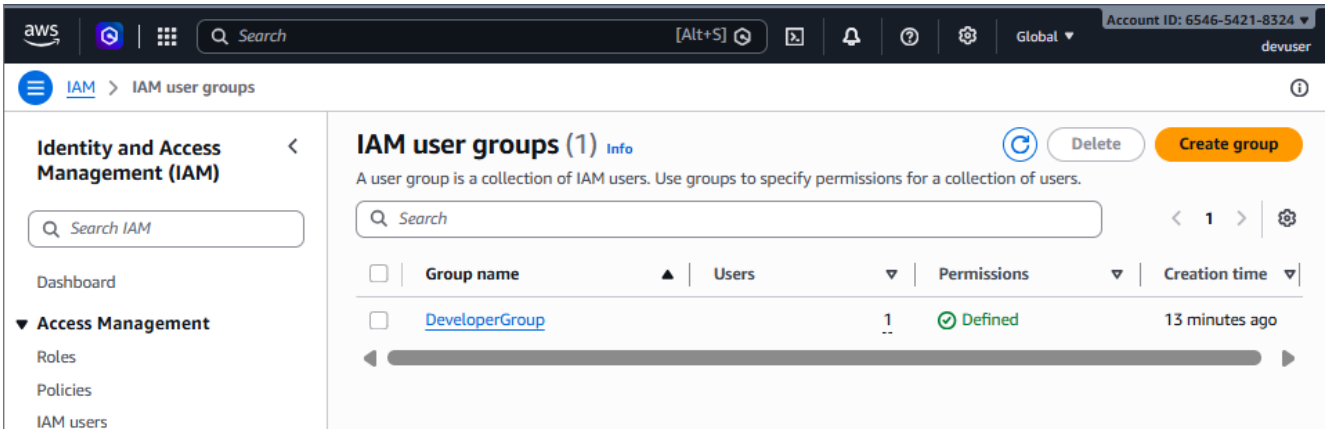


Figure 21. DeveloperGroup is visible after returning to devuser.

Role assumption is therefore not a simple union of all original group permissions and all role permissions. The current session context matters. The report uses these specific observed IAM results rather than generalizing that all IAM access is either present or absent.

15. Inspecting the role and its identity policies

Back as devuser, I searched the role list and opened BucketsAccessRole. The captured role page lists the policies used in the exercise. ListAllBucketsPolicy explains why the role can see bucket names; GrantBucket1Access is the policy discussed in the note for bucket1 reads.

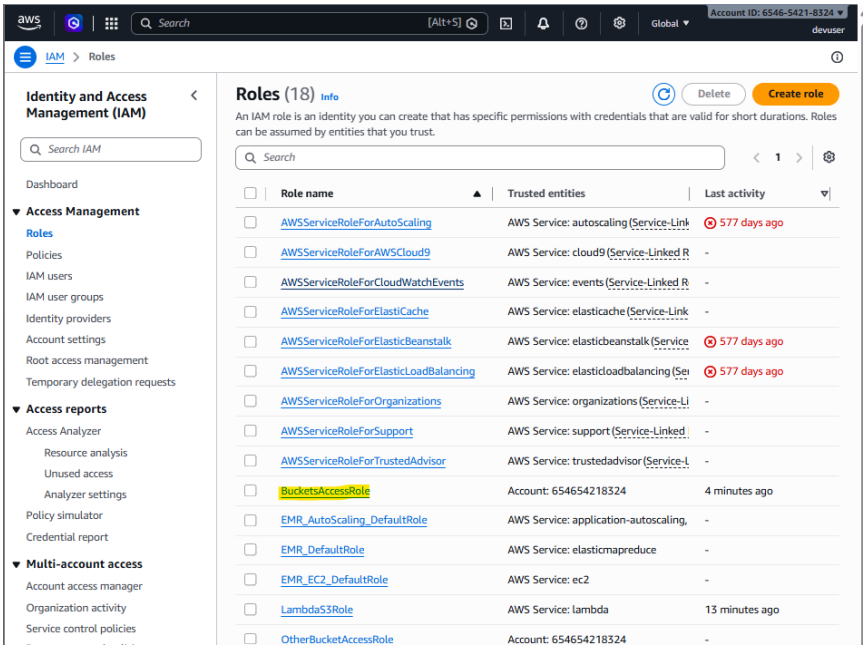


Figure 22. Original role list, including BucketsAccessRole and the separate bucket-access role.

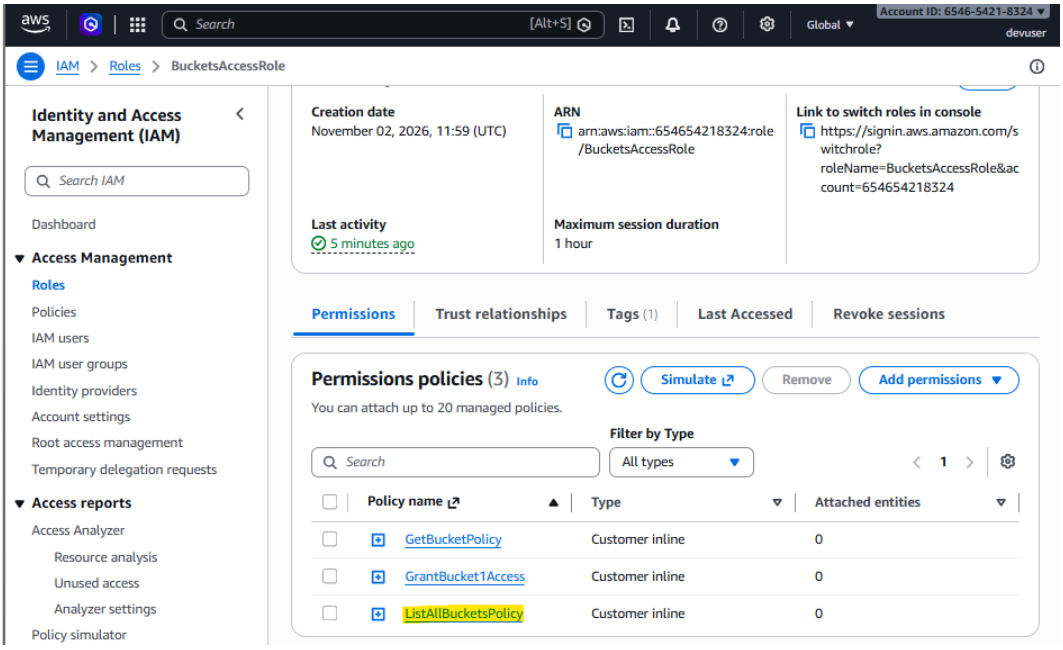


Figure 23. Original role-permissions page listing the policies attached to BucketsAccessRole.

The working note describes ListAllMyBuckets on all resources and object-read/listing access limited to bucket1. It does not establish PutObject permission on bucket1. The raw note also uses the label s3:ListObjects; the documented policy action for the ListObjects API is s3:ListBucket [1].

16. Role credentials and the STS explanation

The following original infographic was added to the notes to explain role assumption. It is an explanatory figure, not a captured exchange of real credentials. It distinguishes the role definition, AWS STS and the temporary credentials used for later requests.

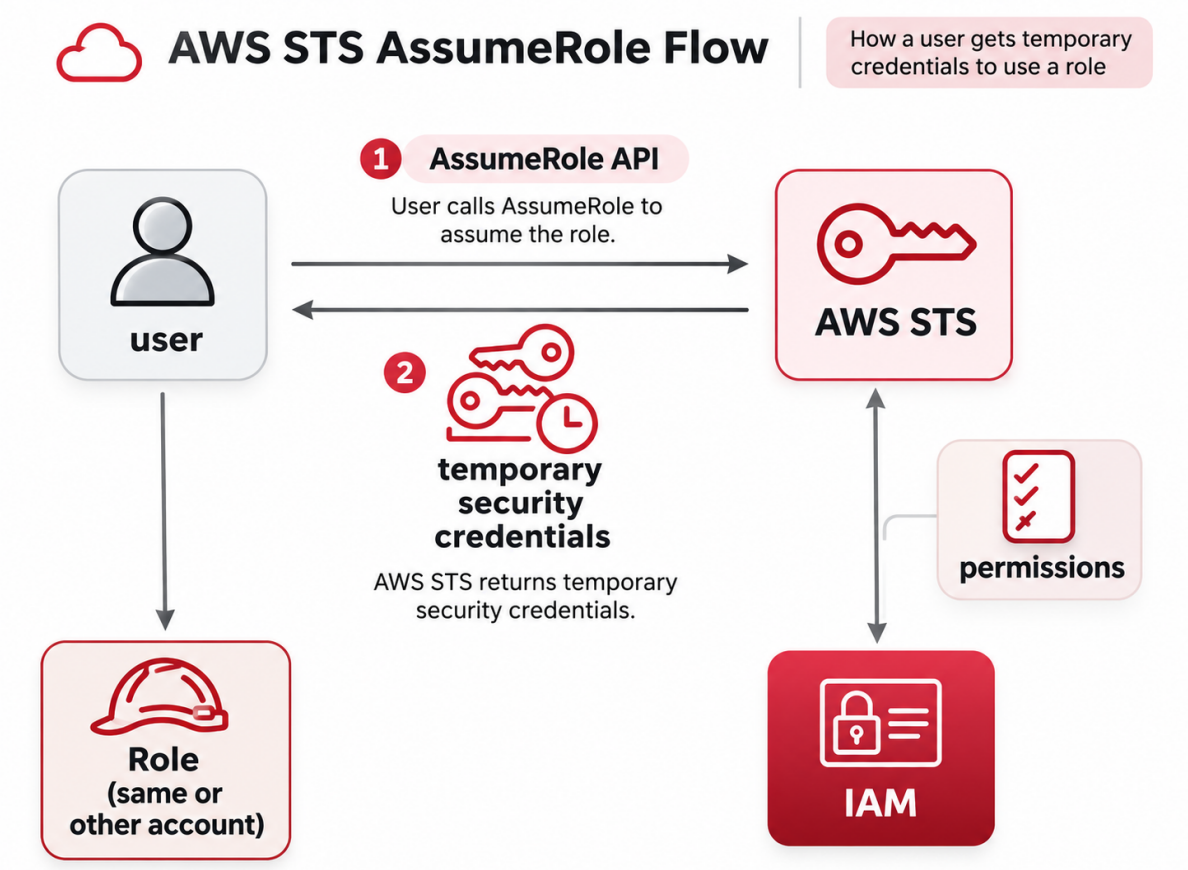


Figure 24. Original STS AssumeRole infographic retained without image changes.

The sequence in the note is: an IAM principal requests to assume a role; the request is evaluated against applicable permissions and the role trust relationship; STS returns temporary credentials; subsequent service requests use the role-session context. No actual access key, secret key or session token was exported as a test artifact for this report.

A trust policy describes who may assume the role. The permissions attached to the role describe permitted work after assumption, subject to other applicable policy limits. The role does not carry every permission inherited by the original IAM user [2, 3].

For bucket1, the explanation supported by the notes is an identity-policy path to GetObject and listing. For bucket2, I needed to inspect the resource side of the relationship. That was the next question raised by the successful access test.

17. Uploading Image2.jpg to bucket2

The first bucket2 capture shows Image2.jpg in the object list under the assumed role. The later capture provides a direct Upload succeeded banner, together with the active BucketsAccessRole label and Image2.jpg in the bucket. These are distinct training sessions, not two consecutive screenshots of one unchanged account.

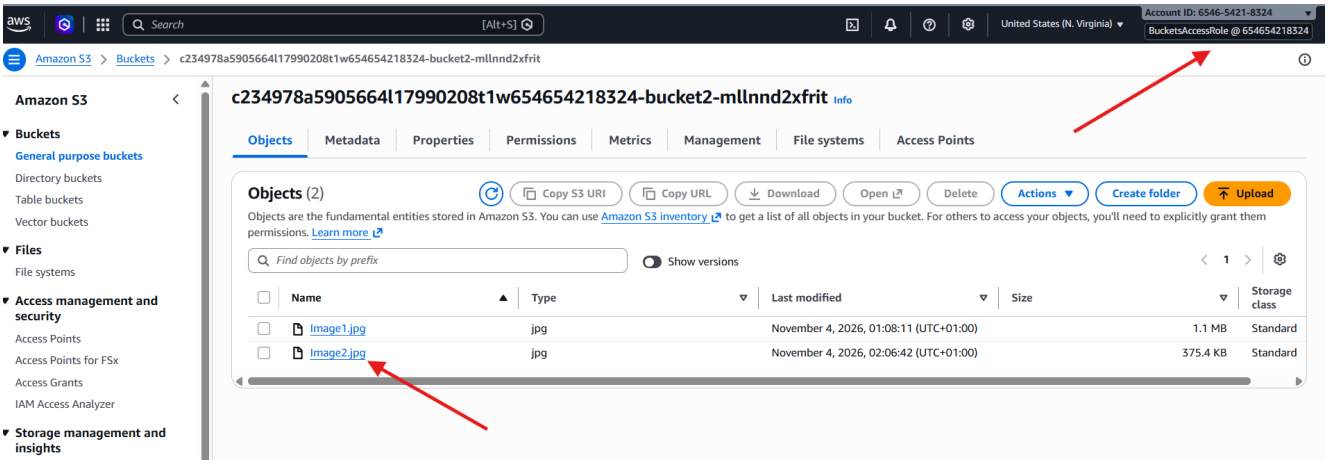


Figure 25. Earlier bucket2 object-list capture under BucketsAccessRole, with Image2.jpg identified.

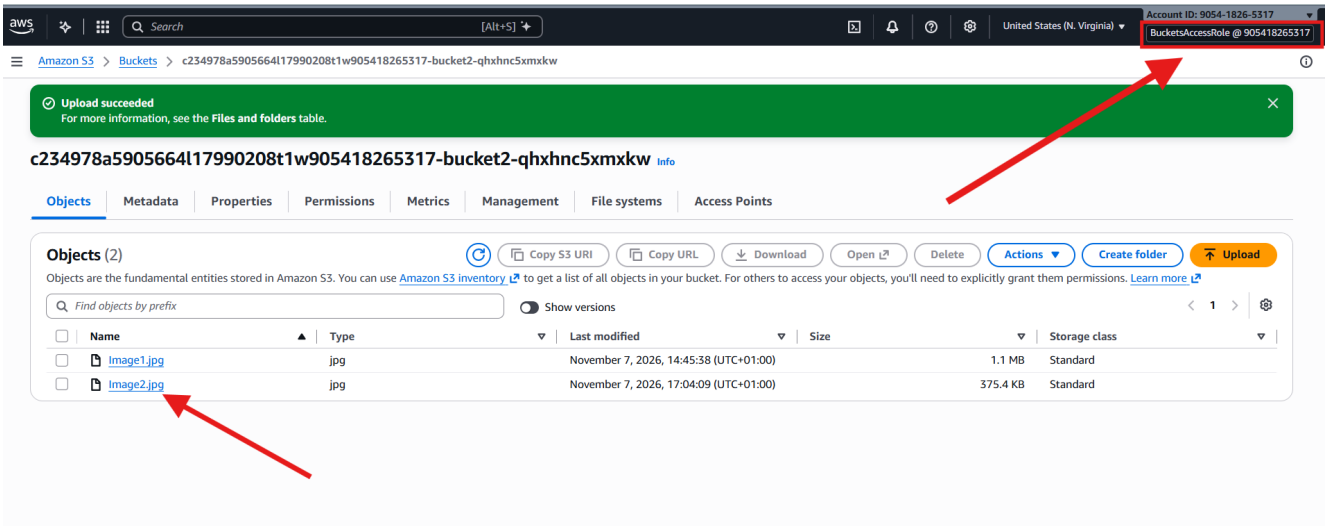


Figure 26. Later bucket2 upload-success capture and object list under BucketsAccessRole.

The written report describes downloading Image2.jpg from bucket1 and uploading it to bucket2. That result initially seemed inconsistent with an identity policy limited to bucket1. The useful question was not whether the role had general S3 access, but which policy could allow PutObject on bucket2.

18. Bucket2 resource policy: explaining the result

I opened bucket2 Permissions and inspected the bucket policy. The first statement, S3Write, names BucketsAccessRole and permits GetObject and PutObject on the bucket2 object resource. The separate ListBucket statement uses the bucket resource. The original policy image is reproduced below, unchanged.


```
{
  "Version": "2008-10-17",
  "Statement": [
    {
      "Sid": "S3Write",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::654654218324:role/BucketsAccessRole"
      },
      "Action": [
        "s3:GetObject",
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::c234978a5905664l17990208t1w654654218324-bucket2-mllnnd2xfrit/*"
    },
    {
      "Sid": "ListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::654654218324:role/BucketsAccessRole"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::c234978a5905664l17990208t1w654654218324-bucket2-mllnnd2xfrit"
    }
  ]
}
```


Figure 27. Original bucket2 policy capture showing the role principal and separate object-access and bucket-listing statements.

The object permission is important: the reviewed role identity policy covered bucket1, but the bucket2 resource policy supplied another access path. For same-account requests, an applicable resource-policy allow can authorize access absent from an identity policy, while an applicable explicit deny overrides an allow. Other limiting controls still need consideration [4].

This is a policy explanation supported by the earlier captured account. The later bucket2 upload screenshot demonstrates a repeated successful outcome. It is not used to claim that the entire later account policy state was exported or independently replayed.

19. Comparing identity and resource access paths

These two original explanatory diagrams were used in the working note to move from the observation to its explanation. The first emphasizes the question raised by bucket2 access. The second labels the role-policy path to bucket1 and the bucket-policy path to bucket2.

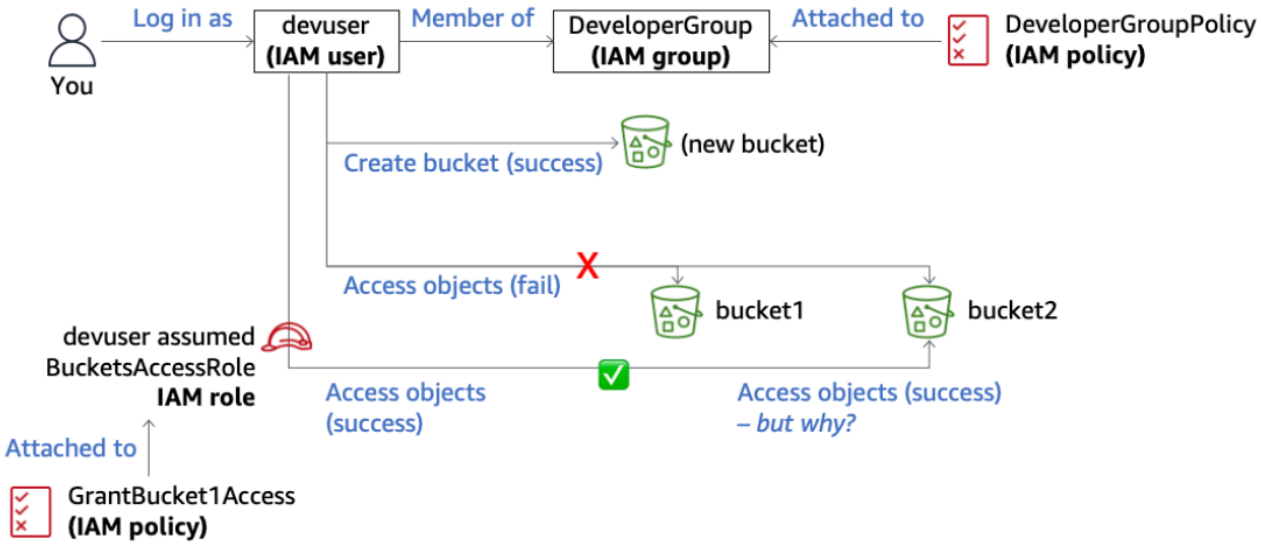


Figure 28. Original explanatory diagram showing bucket1 access and the question raised by bucket2 access.

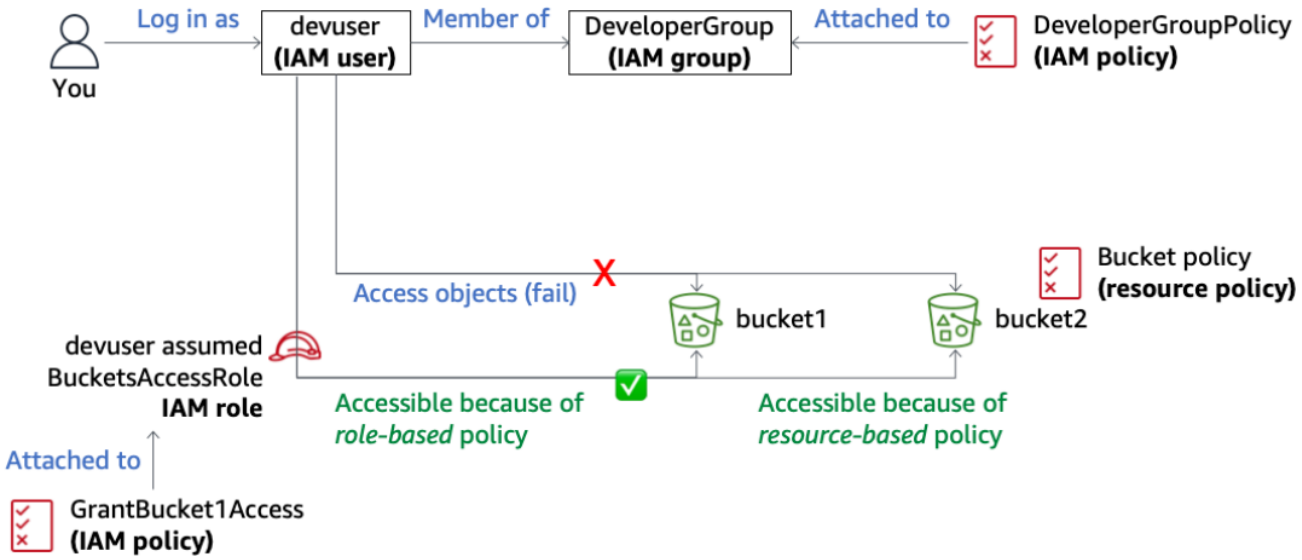


Figure 29. Original diagram separating the role-based and resource-based access paths.

The diagrams help explain the observed workflow, but their arrows are not independent tests. The strongest result statements remain those supported by the Console captures and the inspected policy. I do not infer additional object operations solely from a diagram.

20. Original bucket-access flow illustrations

Both of the following original illustrations are retained as requested. They are explanatory graphics from the notes, not screenshots of successful API requests. They portray role access to two buckets and the role of a bucket resource policy.

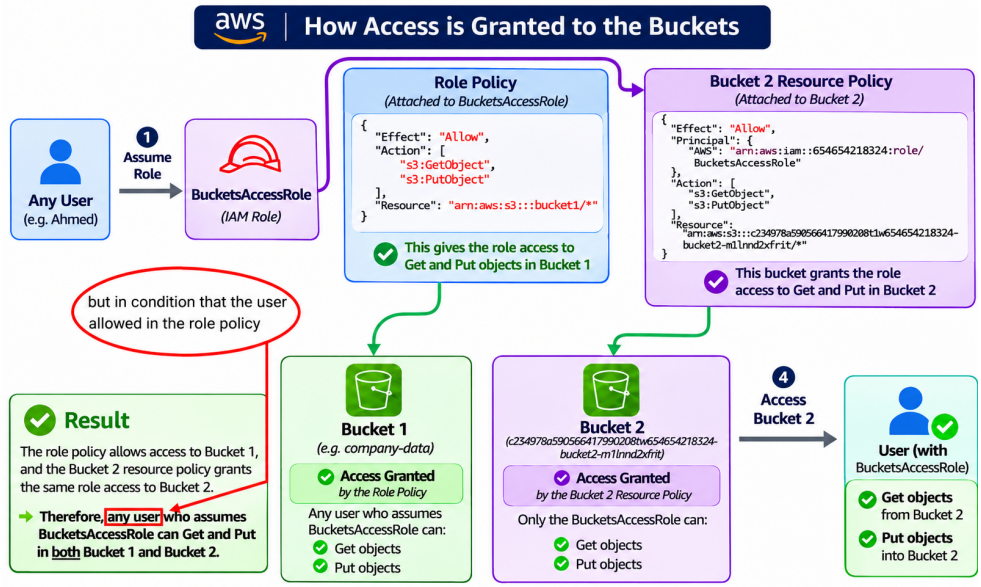


Figure 30. Original bucket-access flow illustration from the notes.

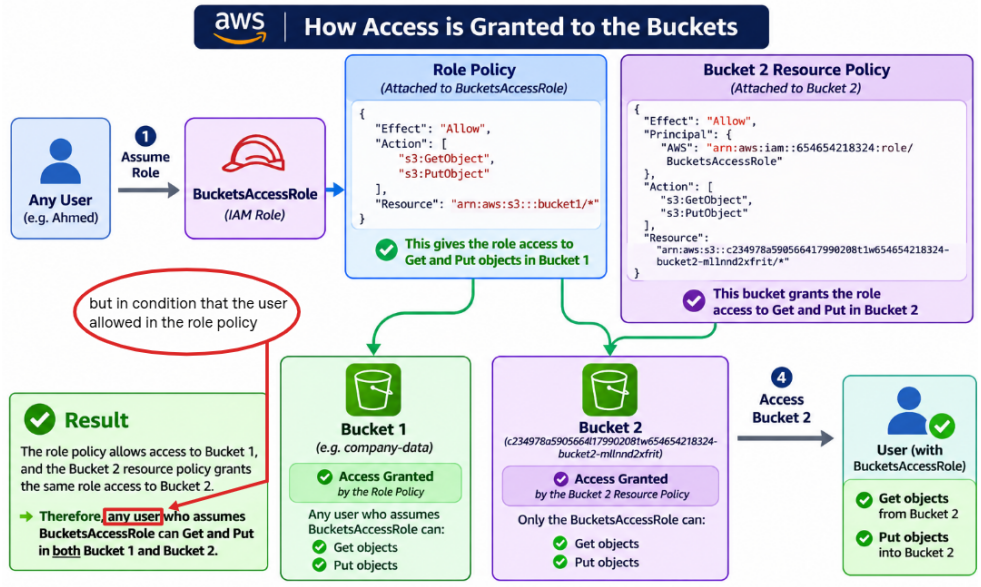


Figure 31. Second original version of the bucket-access flow illustration.

The illustrations label Get and Put access on bucket1. My captured bucket1 test establishes a download, not a verified upload. I therefore do not adopt the graphics as proof of PutObject access to bucket1. This preserves the original images while keeping the written conclusion limited to the evidence.

21. Bucket3 challenge: inspecting the original policy

The challenge asked how to upload Image2.jpg to bucket3. My notes record an initial denial as devuser and an inability to inspect the bucket policy in that context. The first associated image below is actually a group-policy editor view, not a failed upload summary; it should not be mistaken for direct proof of that upload outcome.

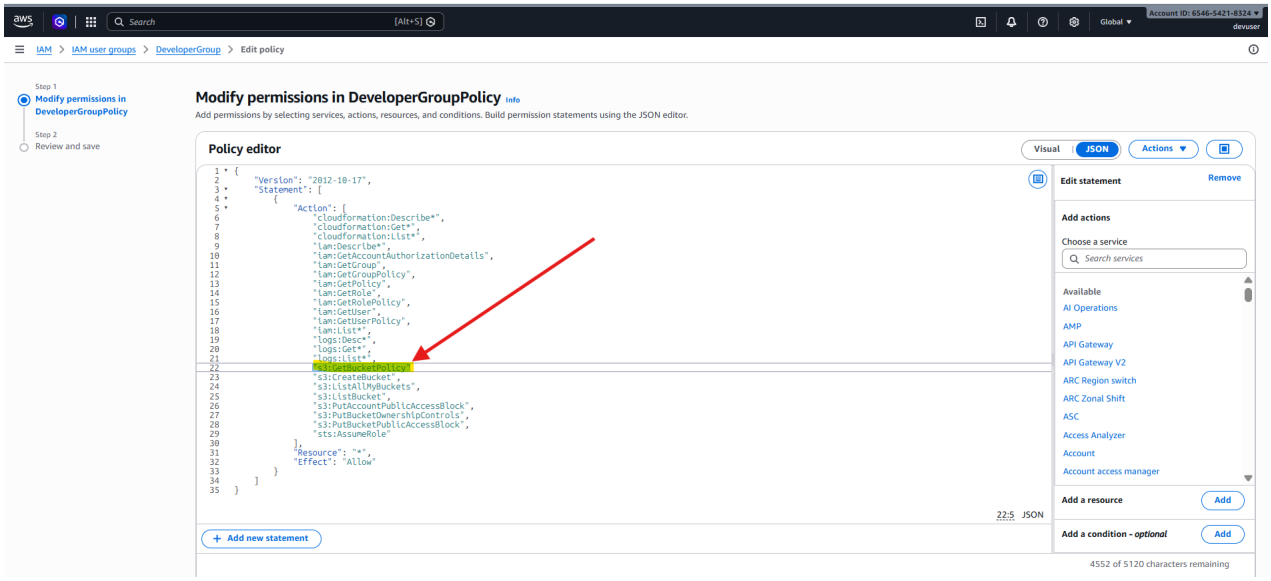


Figure 32. Original group-policy review capture associated with the initial bucket3 investigation.

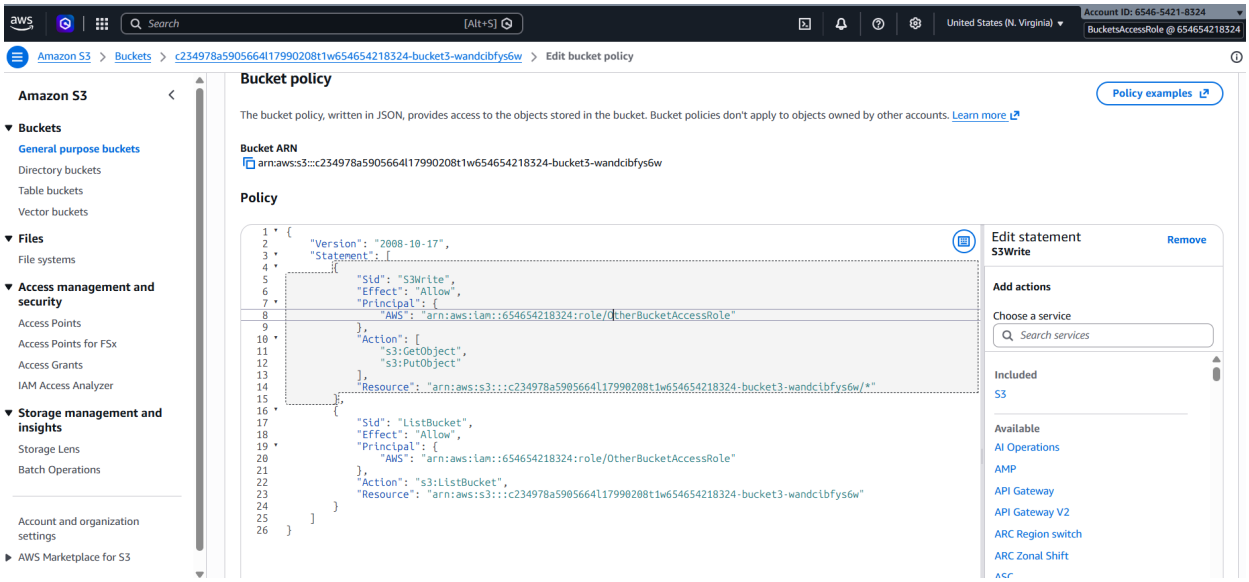


Figure 33. Earlier bucket3 policy-editor capture naming a different bucket-access role.

The important clue was the Principal in the bucket3 resource policy. It named a different role from BucketsAccessRole. That observation suggested testing the role identified by the resource, rather than assuming that every role capable of reading bucket1 could also write to bucket3.

22. Bucket3: policy principal and successful upload

The newer annotated policy capture identifies OtherBucketAccessRole as the bucket3 principal. The subsequent upload summary shows that role active, Image2.jpg marked Succeeded, one successful file and no failed files. The destination identifier matches the bucket3 policy capture from this later training account.

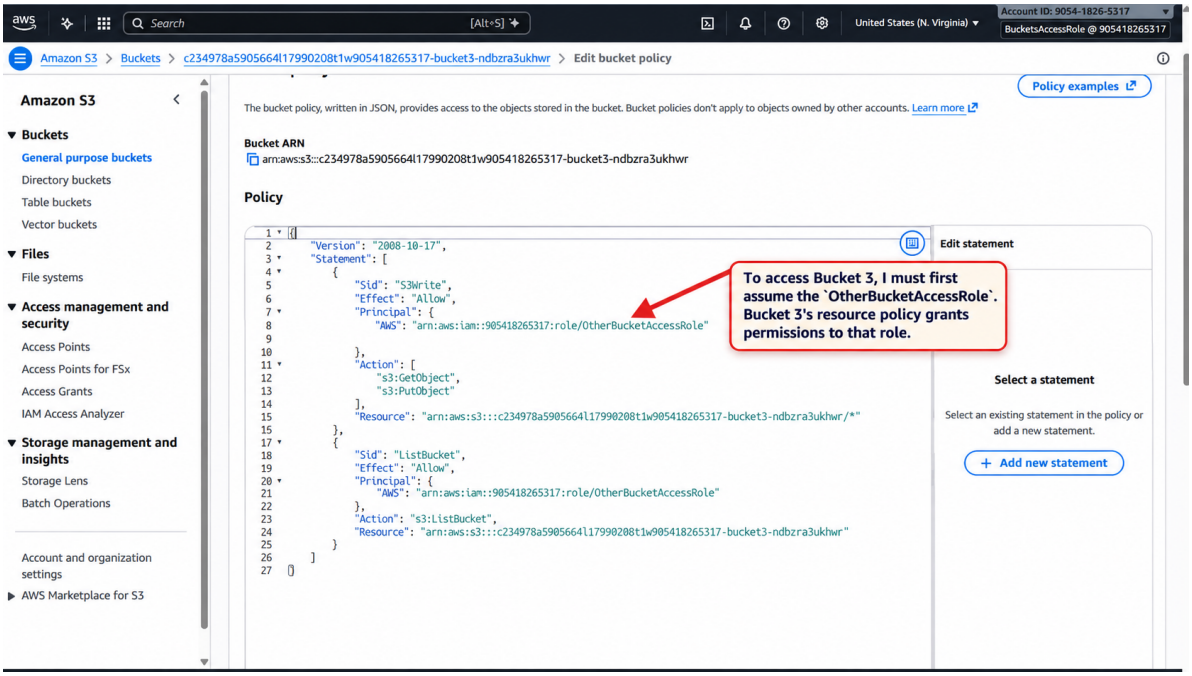


Figure 34. Original annotated bucket3 policy capture pointing to OtherBucketAccessRole.

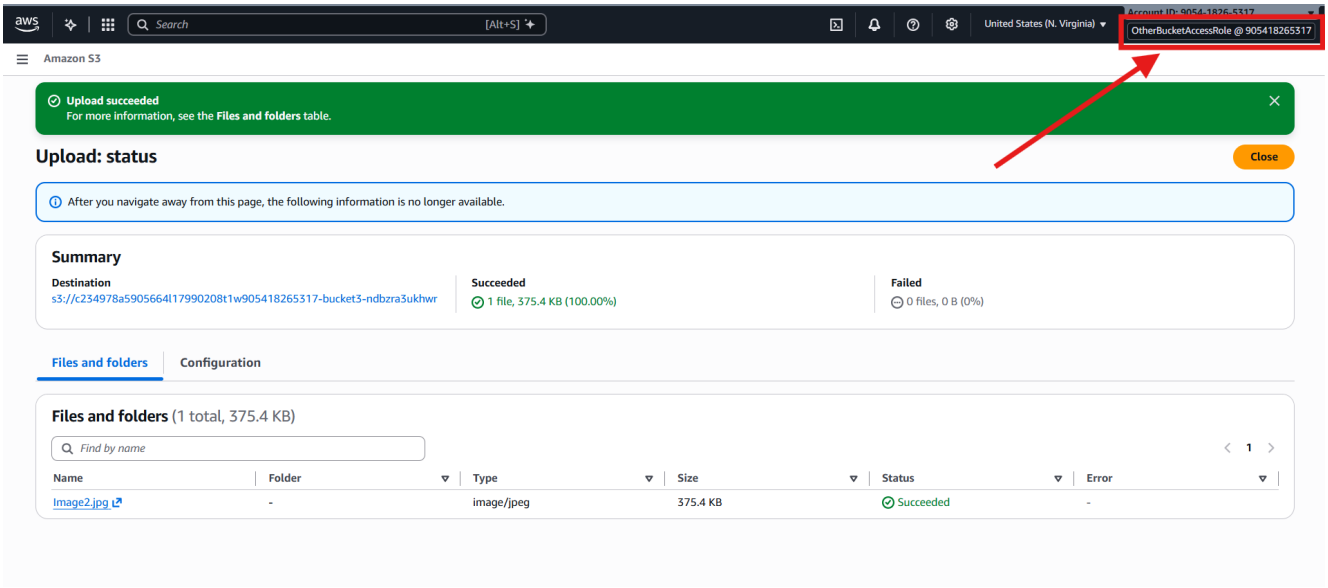


Figure 35. Original bucket3 upload-success capture under OtherBucketAccessRole.

This newer evidence changes the older report: the captured bucket3 upload is supported as successful. The statement remains narrow. The record does not include a complete role-assumption policy audit for OtherBucketAccessRole or an unseen final grade for the training exercise.

23. Technical discussion and practical lessons

Action and resource are separate parts of the authorization question. CreateBucket creates a bucket; PutObject writes an object; GetObject reads an object. Being allowed to create or list a bucket does not automatically authorize every operation on its contents. My create-then-upload sequence made that distinction visible.

Bucket and object resources also differ. A listing permission uses the bucket ARN, while object actions address objects, commonly through a bucket object pattern ending in /*. The source note uses s3:ListObjects in one policy example; the explanation here follows the documented s3:ListBucket policy action for the ListObjects API [1]. I do not present that raw note excerpt as a production-ready policy.

The current principal must be checked before explaining a result. The IAM group-list comparison showed an operation available to devuser but denied after assuming BucketsAccessRole. In the role-session context, the original user permissions are not simply added to the role permissions. STS provides temporary credentials, and applicable role/session and resource policies govern the later requests [2, 3].

Identity policies are only one side of an S3 access review. The supplied role policy explained bucket1 reads; bucket2 inspection revealed a resource policy naming the role. In the same-account case studied here, an applicable allow in either relevant identity or resource policy can matter. An explicit deny takes precedence, and permissions boundaries or other limiting policy types can affect the result [4].

Trust and access permissions should not be confused. The trust relationship explains who may assume a role. The S3 identity or resource policy explains the particular operation after assumption. Seeing a trusted user is not a blanket permission grant for every service.

The bucket3 investigation reinforced the need to read the exact resource-policy principal. The successful capture used OtherBucketAccessRole rather than treating BucketsAccessRole as a universal bucket-access role. I kept the separate sessions, the supplied configurations and the observed results distinct.

For a production review I would extend this work to explicit denies, Organizations controls, permissions boundaries, session policies, encryption-related permissions and bucket settings. Those checks are future work, not claims made from the current lab captures.

24. Limitations, next steps and references

The evidence comes from private working notes and Console captures taken across more than one training session. Account identifiers and dates differ. I have not combined them into a fictional continuous test run. The formal results distinguish a screenshot, a written observation and an explanatory illustration.

The later bucket3 capture supports one successful upload. The full assumption-policy chain for OtherBucketAccessRole, a full effective-permission export, the final training grade, and CloudTrail evidence are not established by the source material. The example account-summary output included later is illustrative and is not claimed as a command I executed.

The supplied identities, groups, policies, roles and initial buckets belong to the training setup. My documented work was exploring access, switching roles, testing requests, reading policies and explaining the results. The exercise instructions themselves are not reproduced as a full course handout.

A stronger repeatable test would record the active identity and the API outcomes within one controlled session, keep the exact relevant policy exports, and associate each request with its principal, resource and result. That would make it easier to separate an implicit denial, an explicit denial and other possible restrictions.

For future work I would also verify the permissions for OtherBucketAccessRole and repeat the challenge from a recorded starting state. I would preserve unsuccessful tests as well as successful ones; the errors explain the boundaries more clearly than a final success screen alone.

All 38 original images are included without alteration. Their original filenames are retained in assets/ and indexed at the end of this report. The original Obsidian notes and image files remain unchanged.

References

[1] Amazon S3: Required permissions for Amazon S3 API operations.
https://docs.aws.amazon.com/AmazonS3/latest/userguide/using-with-s3-policy-actions.html

[2] AWS STS: AssumeRole.
https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRole.html

[3] AWS IAM: Permissions for AssumeRole and related operations.
https://docs.aws.amazon.com/IAM/latest/UserGuide/id_credentials_temp_control-access_assumerole.html

[4] AWS IAM: Policy evaluation logic.
https://docs.aws.amazon.com/IAM/latest/UserGuide/reference_policies_evaluation-logic.html

Appendix A. Additional original context captures

These original images were present alongside the main workflow and are retained without changes. The first is the unannotated version of the later active-role Console view. The second is an earlier conceptual summary of user access: bucket creation succeeds while object access is unavailable.

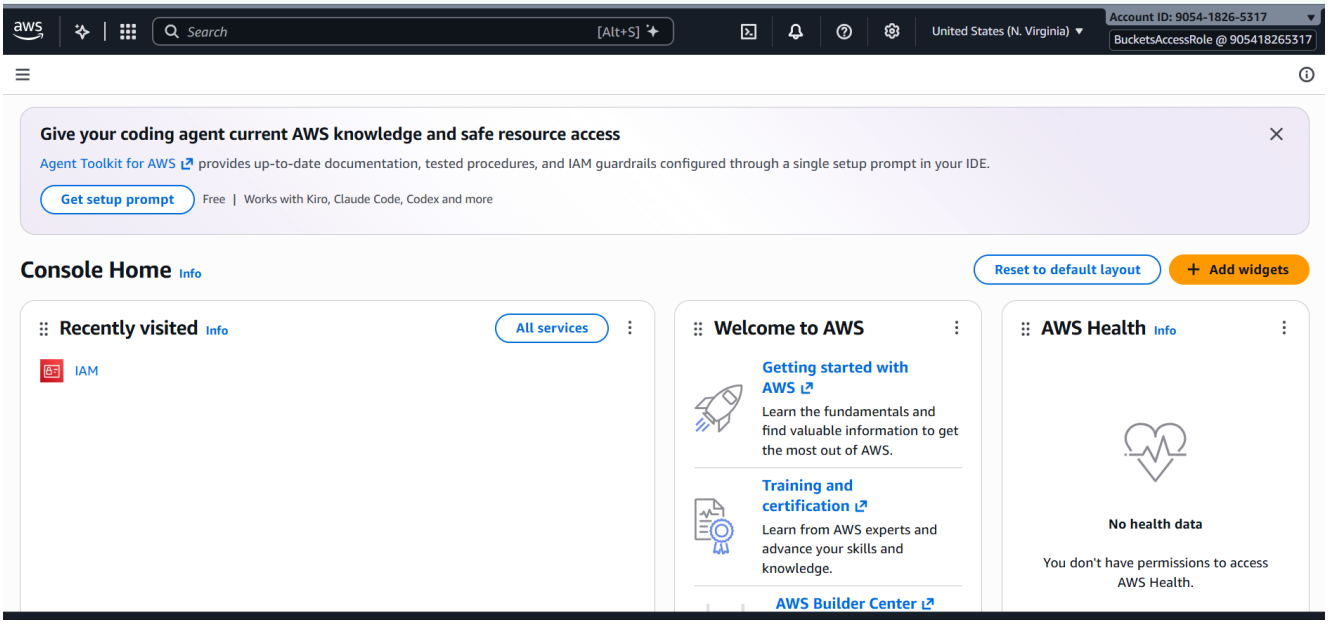


Figure 36. Additional original Console view with BucketsAccessRole active.

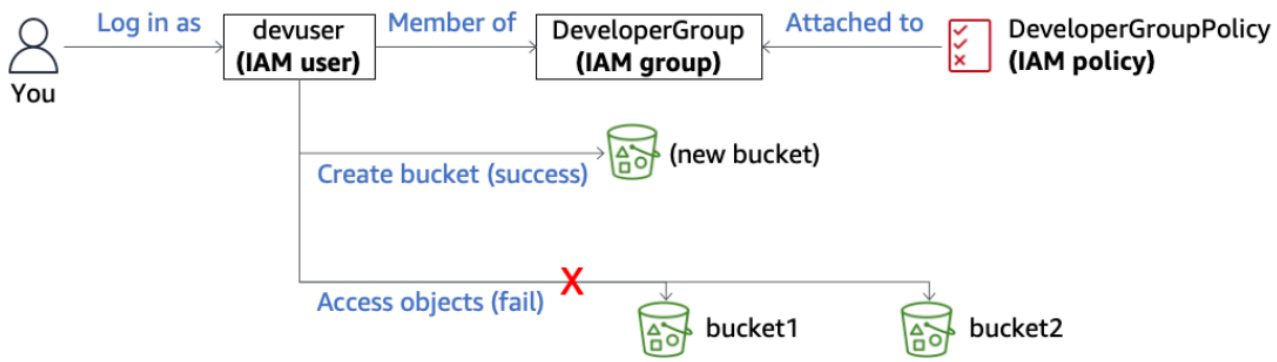


Figure 37. Original conceptual user-access diagram retained with the object-access discussion.

These images add context and preserve the complete visual record. They are not treated as new independent operations merely because the source folder contains both annotated and unannotated versions.

Appendix B. Illustrative account-summary command

The working note includes the following image of aws iam get-account-summary and an example JSON response. It was used to explain what the denied IAM dashboard API would return when allowed. The source does not establish that I executed this command or obtained these exact counts from the training account.

```
aws iam get-account-summary
```

Output:


```
{
    "SummaryMap": {
        "UsersQuota": 5000,
        "GroupsQuota": 100,
        "InstanceProfiles": 6,
        "SigningCertificatesPerUserQuota": 2,
        "AccountAccessKeysPresent": 0,
        "RolesQuota": 250,
        "RolePolicySizeQuota": 10240,
        "AccountSigningCertificatesPresent": 0,
        "Users": 27,
        "ServerCertificatesQuota": 20,
        "ServerCertificates": 0,
        "AssumeRolePolicySizeQuota": 2048,
        "Groups": 7,
        "MFADevicesInUse": 1,
        "Roles": 3,
        "AccountMFAEnabled": 1,
        "MFADevices": 3,
        "GroupsPerUserQuota": 10,
        "GroupPolicySizeQuota": 5120,
        "InstanceProfilesQuota": 100,
        "AccessKeysPerUserQuota": 2,
        "Providers": 0,
        "UserPolicySizeQuota": 2048
    }
}
```


Figure 38. Original illustrative GetAccountSummary command/output image, included as explanatory material rather than a verified CLI transcript.

This distinction is important: an example output can explain an API, but it does not overturn the actual Console denial shown earlier. I therefore retain the image while describing its evidential role accurately.

Appendix C. Complete original-image index

Every image below is included in this PDF and copied unchanged to assets/. The figure numbers identify where each original appears; the filenames match the source lab folder.

Figure	Original image filename
30	AWS Bucket Access Flow.png
34	AWS S3 Bucket Policy Role Access Callout.png
24	AWS STS AssumeRole Flow Infographic.png
8	Pasted image 20261107155958.png
10	Pasted image 20261107160314.png
11	Pasted image 20261107160340.png
12	Pasted image 20261107160839.png
13	Pasted image 20261107160911.png
14	Pasted image 20261107161045.png
36	Pasted image 20261107161139.png
15	Pasted image 20261107161203.png
16	Pasted image 20261107164854.png
17	Pasted image 20261107164923.png
26	Pasted image 20261107170546.png
35	Pasted image 20261107173319.png
3	S3-Resource-Based-Policies-Lab-20261029174547.png
4	S3-Resource-Based-Policies-Lab-20261029174640.png
1	S3-Resource-Based-Policies-Lab-20261030074647.png
2	S3-Resource-Based-Policies-Lab-20261030074704.png
37	S3-Resource-Based-Policies-Lab-20261030074823.png
28	S3-Resource-Based-Policies-Lab-20261030074911.png
29	S3-Resource-Based-Policies-Lab-20261030074934.png
5	S3-Resource-Based-Policies-Lab-20261030080222.png
6	S3-Resource-Based-Policies-Lab-20261030080612.png
38	S3-Resource-Based-Policies-Lab-20261030081619.png
7	S3-Resource-Based-Policies-Lab-20261030082322.png
9	S3-Resource-Based-Policies-Lab-20261030085619.png
18	S3-Resource-Based-Policies-Lab-20261102130442.png
19	S3-Resource-Based-Policies-Lab-20261102130531.png
20	S3-Resource-Based-Policies-Lab-20261102131112.png
21	S3-Resource-Based-Policies-Lab-20261102131156.png
22	S3-Resource-Based-Policies-Lab-20261102131338.png
23	S3-Resource-Based-Policies-Lab-20261102131517.png
25	S3-Resource-Based-Policies-Lab-20261104020946.png
27	S3-Resource-Based-Policies-Lab-20261104021707.png
31	S3-Resource-Based-Policies-Lab-20261104023951.png
32	S3-Resource-Based-Policies-Lab-20261104024727.png
33	S3-Resource-Based-Policies-Lab-20261104025558.png